

Performance improvements of commonly used optimizers in Machine Learning using **CUDA** or **OpenCL**

Lennart Hahner

Table of Contents

I. Motivation

II. Related Work & Expectations

III. Conceptual differences of OpenCL and CUDA

IV. The Implementation

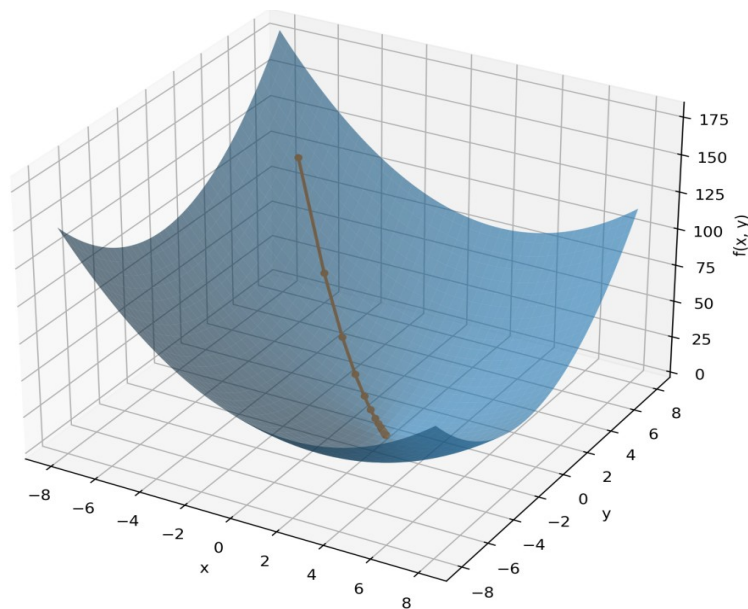
V. Benchmarks



Motivation

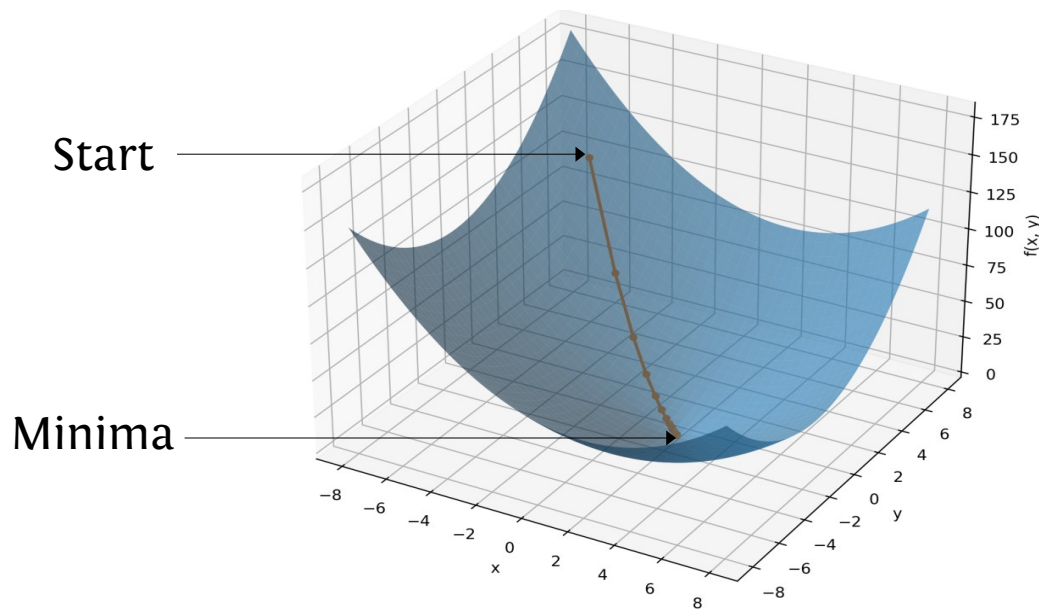
The role of optimization in machine learning

Optimizers are the engines of machine learning models.



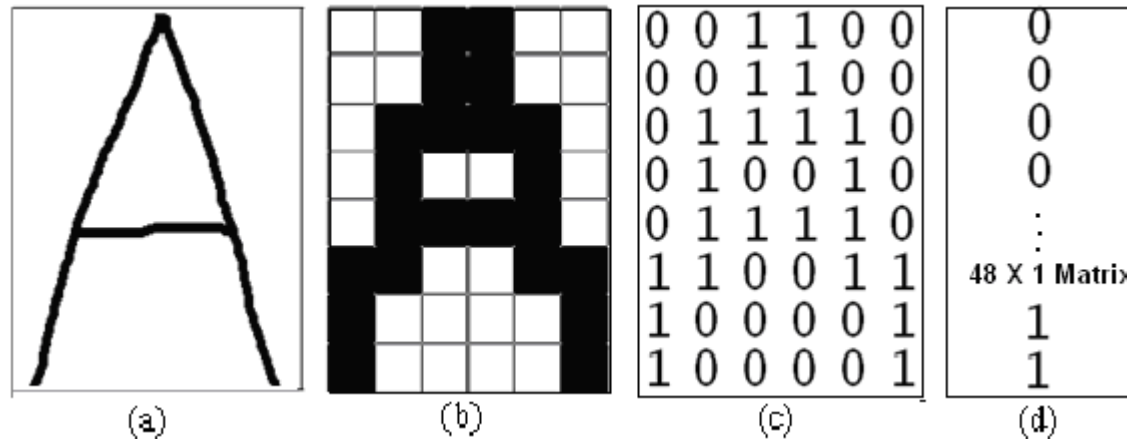
The role of optimization in machine learning

Train learnable weights towards a minimum of a loss function.



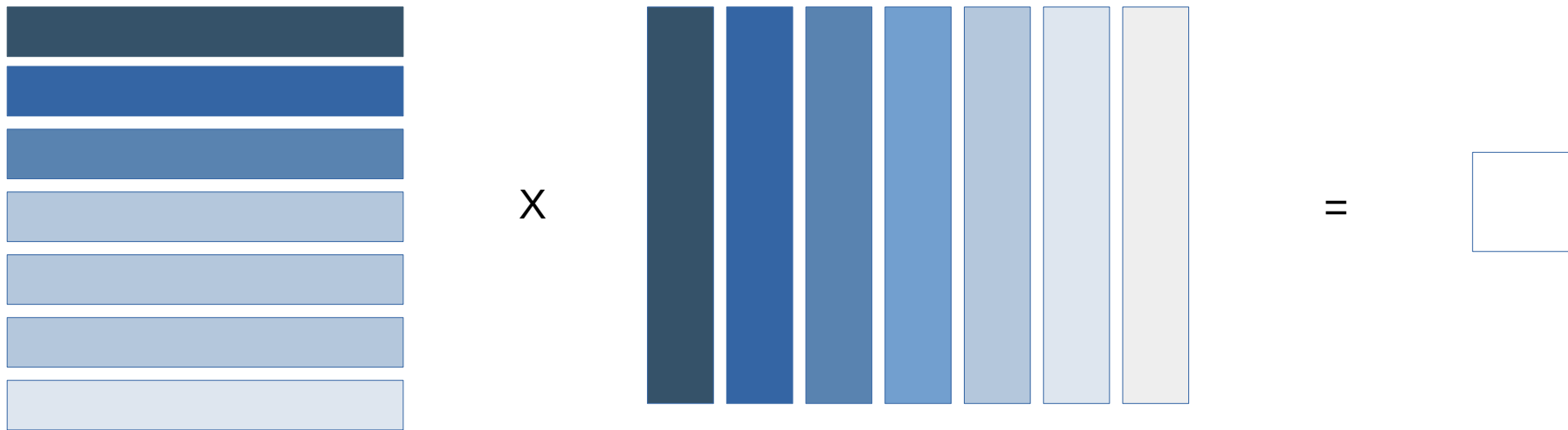
The role of optimization in machine learning

Most inputs i.e. images or text are represented as matrices...



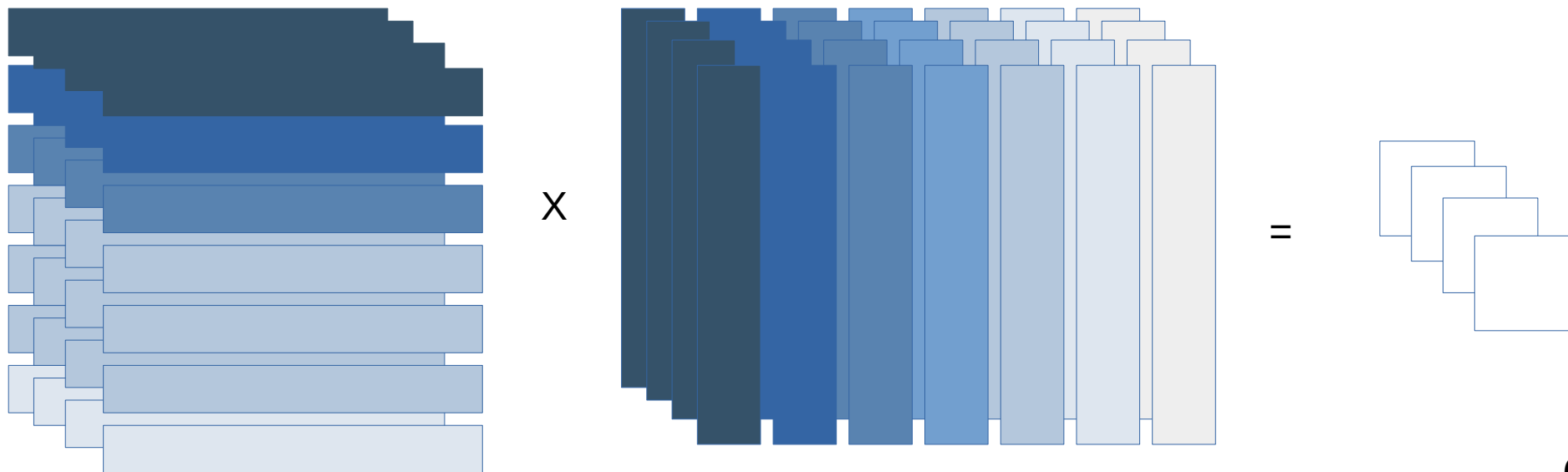
The role of hardware in machine learning

Thus we often deal with **matrix** or tensor operations during optimizations...



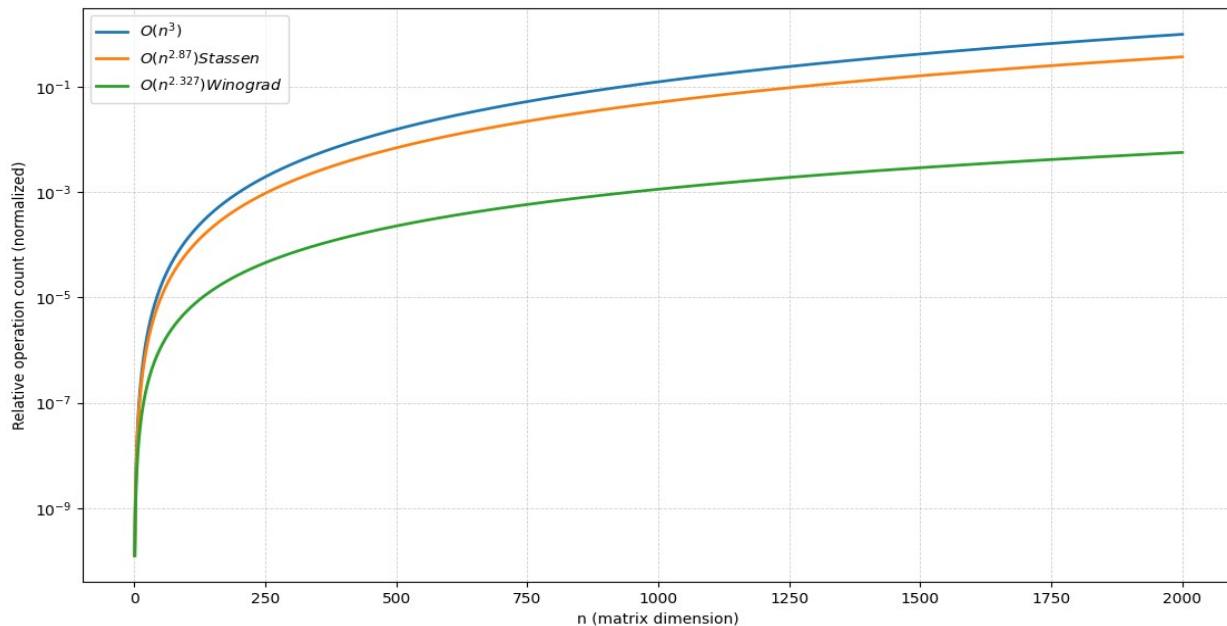
The role of hardware in machine learning

Thus we often deal with matrix or **tensor** operations during optimizations...



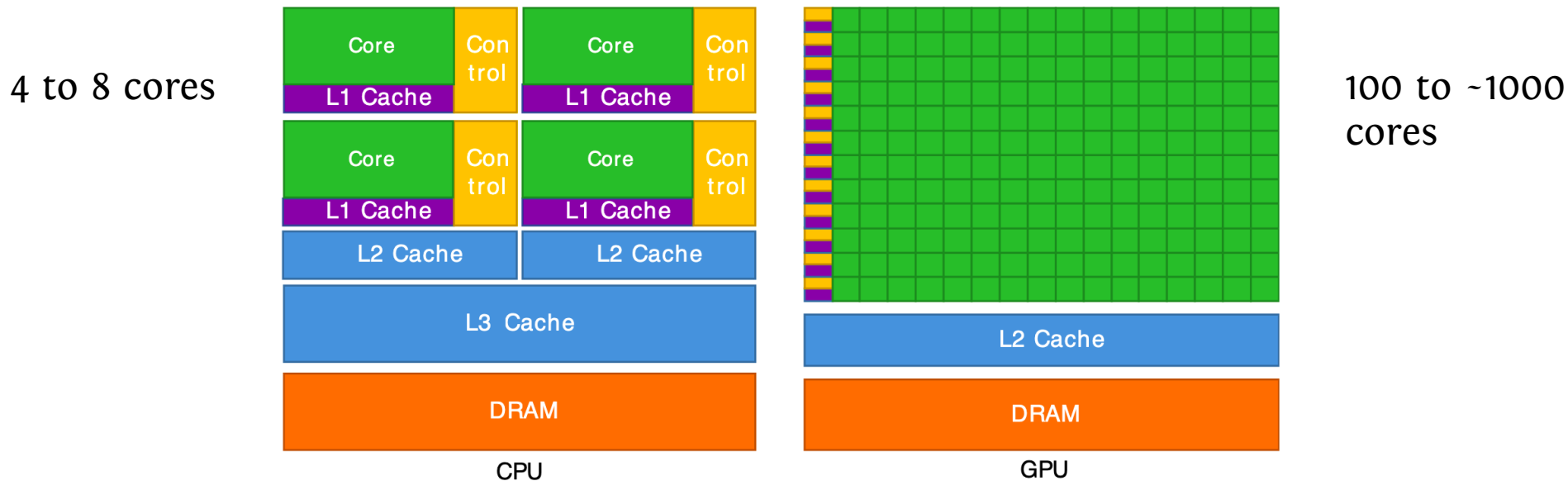
The role of hardware in machine learning

Matrix and Tensor operations are computationally expensive...



The role of hardware in machine learning

...exploiting hardware for parallelization



The role of hardware in machine learning

...how to exploit GPUs?

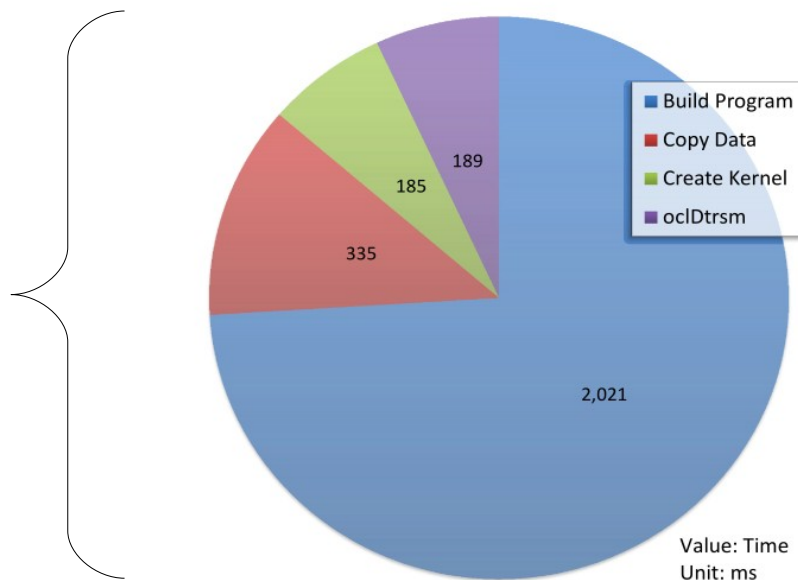


Expectations

Results of „From CUDA to OpenCL“ [2]

OpenCL enables code portability, but not performance

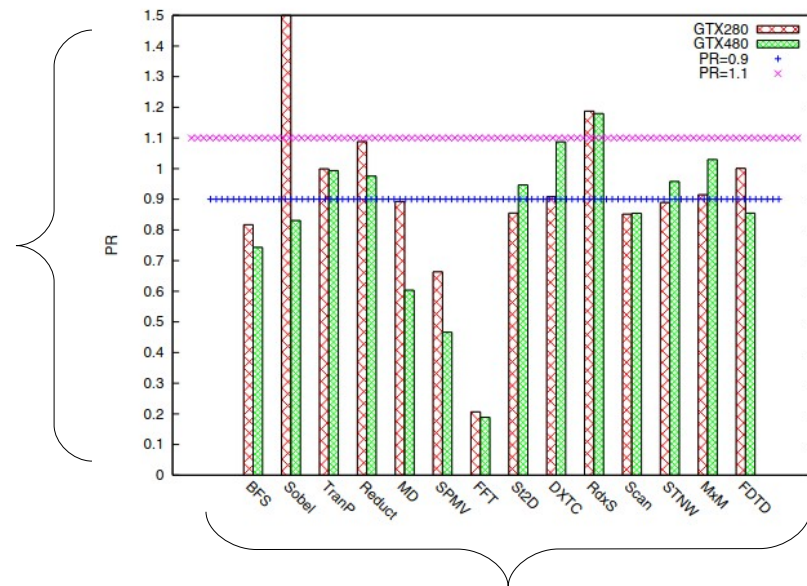
OpenCL GEMM
program



Results of „A Comprehensive Performance Comparision of CUDA and OpenCL“ [3]

CUDA is at most ~30% faster than OpenCL for most applications

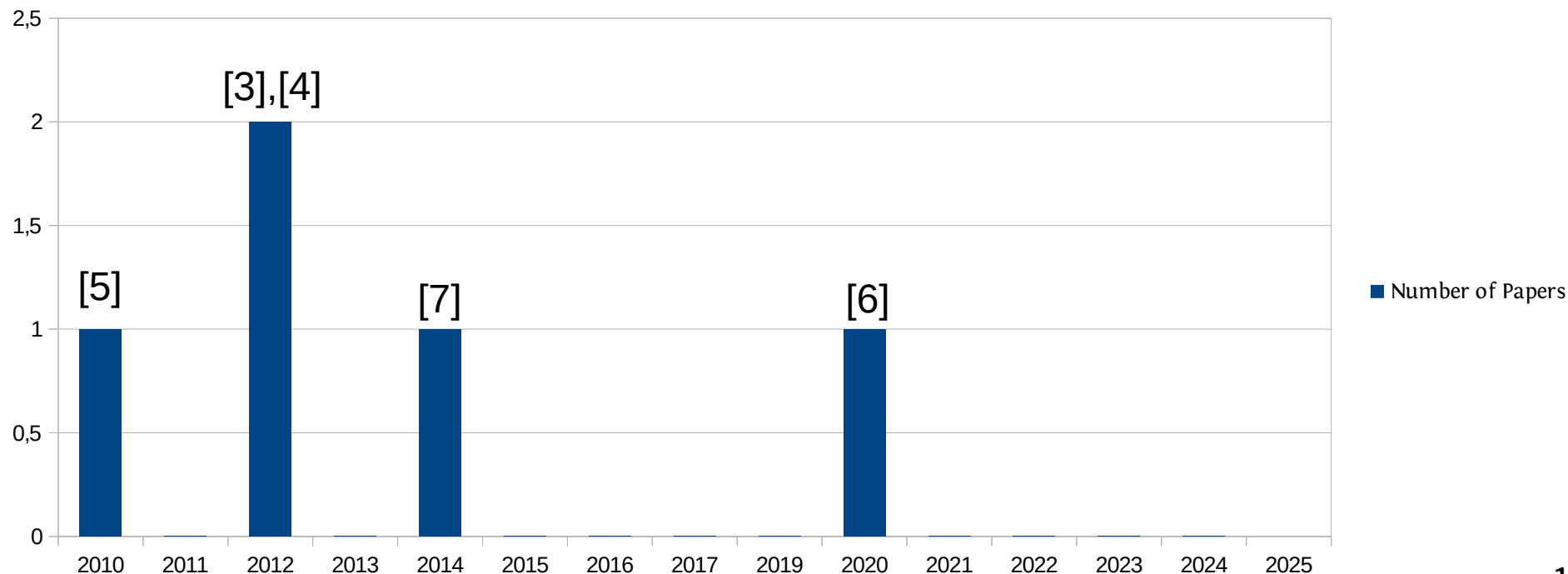
$$PR = \frac{\text{Perf}(\text{OpenCL})}{\text{Perf}(\text{CUDA})}$$



Workloads

About more results

Most of the comparision results are more than 10 years old...



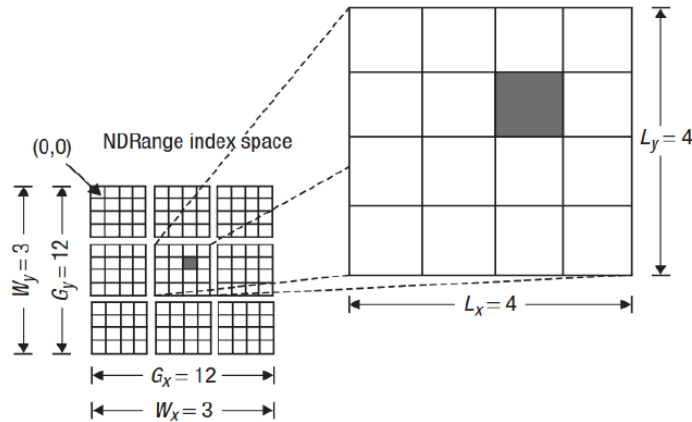
So what to expect from the comparison?

...CUDA might perform better, due to the longer compile/build time of OpenCL, but it's worth revisiting...

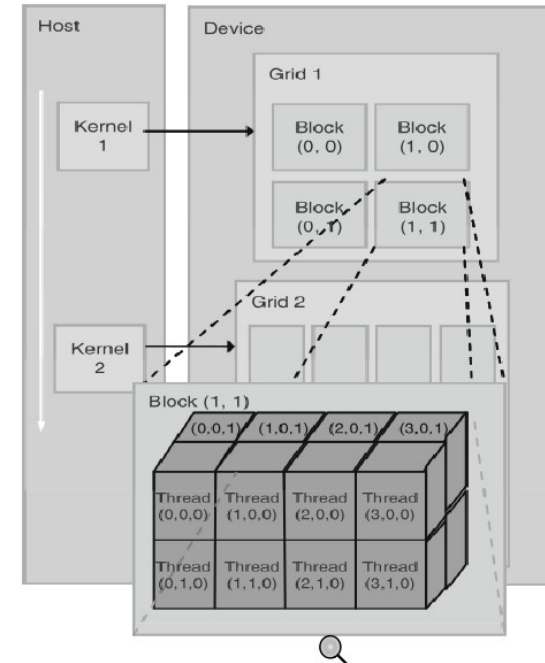
Conceptual Comparison of CUDA and OpenCL



Architectural differences



[8] Figure 1.7 in Aftab M. et al. "OpenCL Programming Guide"



[9] Figure 3.13 in Kirk D. et al. "Programming Massively Parallel Processors"

Language & Usage differences



```
__global__ void adam_update_kernel_impl(  
    float *__restrict__ param,  
    const float *__restrict__ grad,  
    float *__restrict__ m,  
    float *__restrict__ v,  
    size_t n,  
    float lr, float beta1,  
    float beta2,  
    float bc1, float bc2, float eps)  
{  
    ...  
}
```

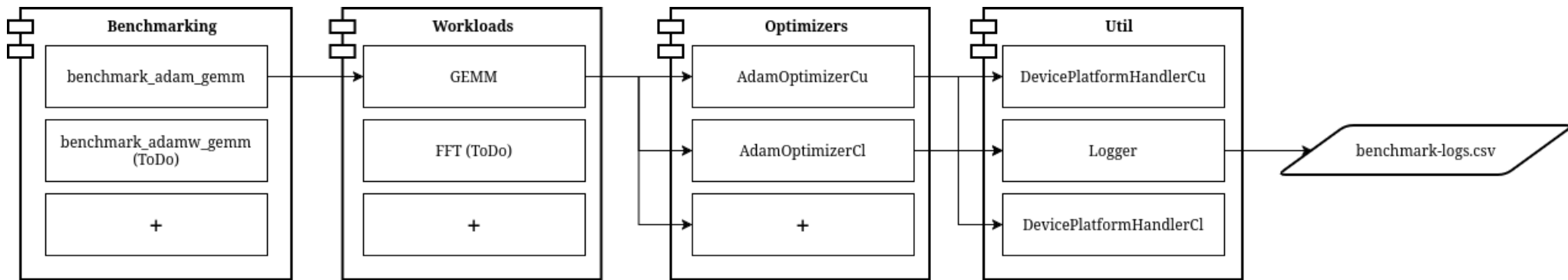


```
__kernel void adam_update(  
    __global float*      param,  
    __global const float* grad,  
    __global float*      m,  
    __global float*      v,  
    const float lr,  
    const float beta1,  
    const float beta2,  
    const float eps,  
    const float bc1,  
    const float bc2,  
    const int  n  
) {  
    ...  
}
```

Benchmark program implementation

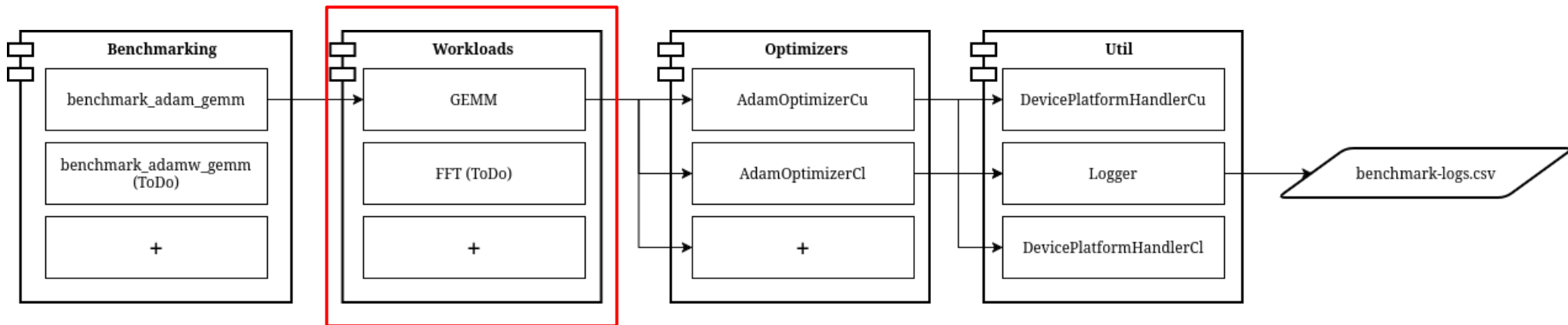
Architecture overview

Target; make it as extendable as possible for further permutations



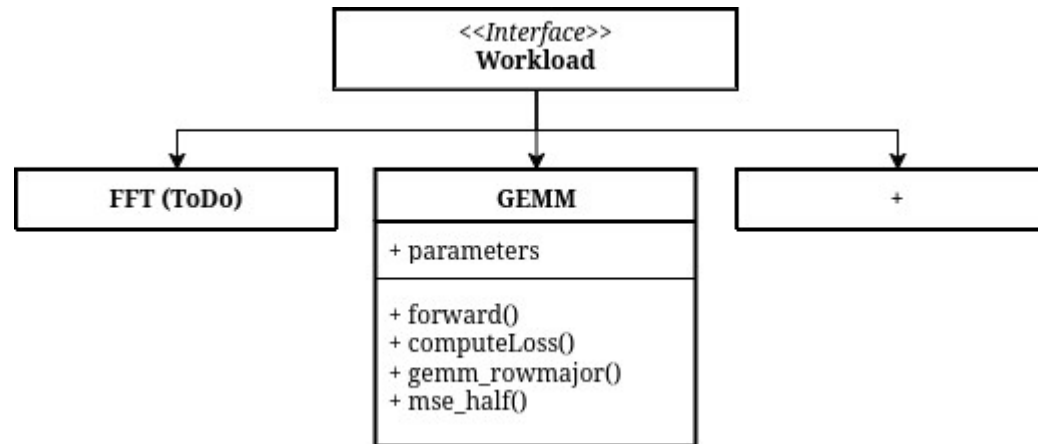
Workload Module

Definition and initialization of workloads...



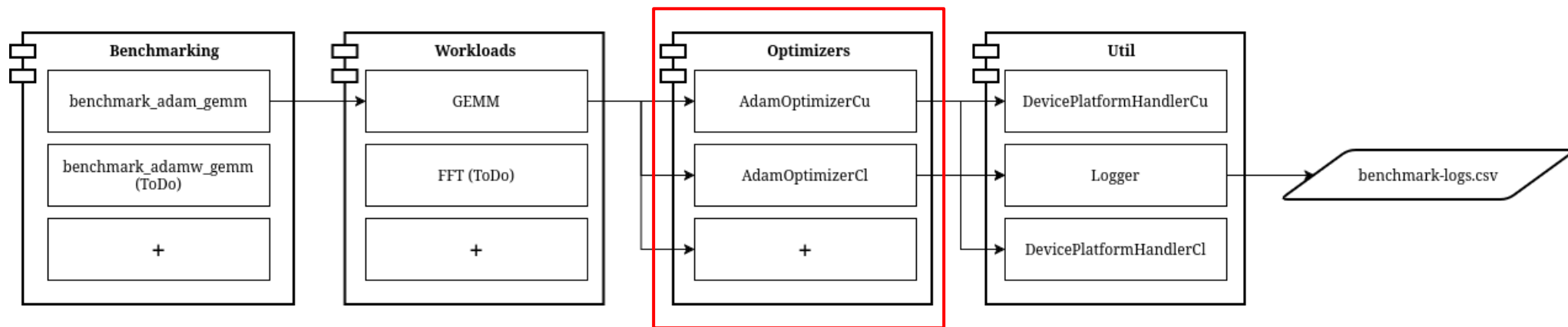
Workload Module

Definition and initialization of workloads...

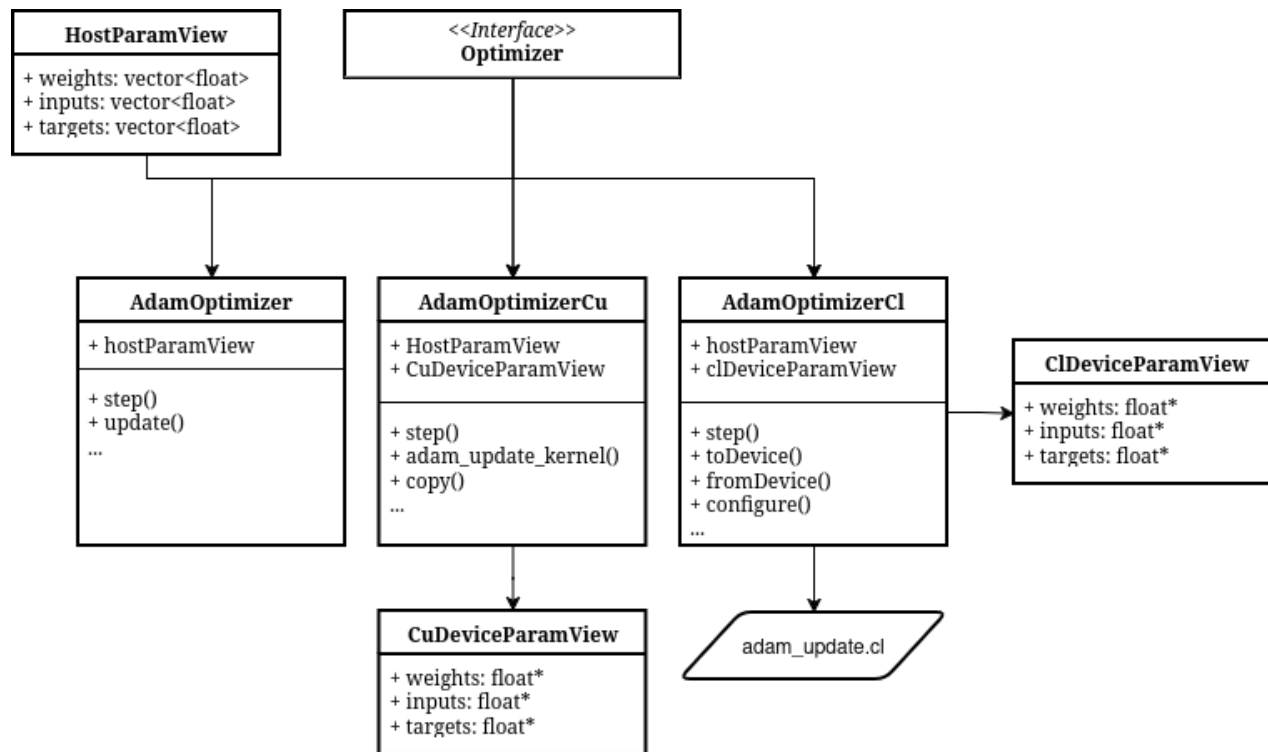


Optimizer Module

The complete implementation of the Optimizer in CUDA and OpenCL

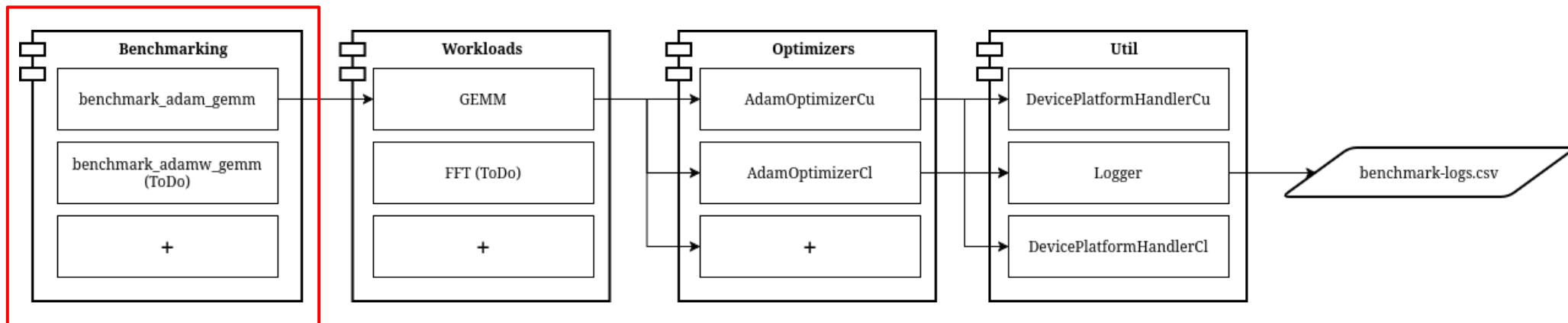


Optimizer Module



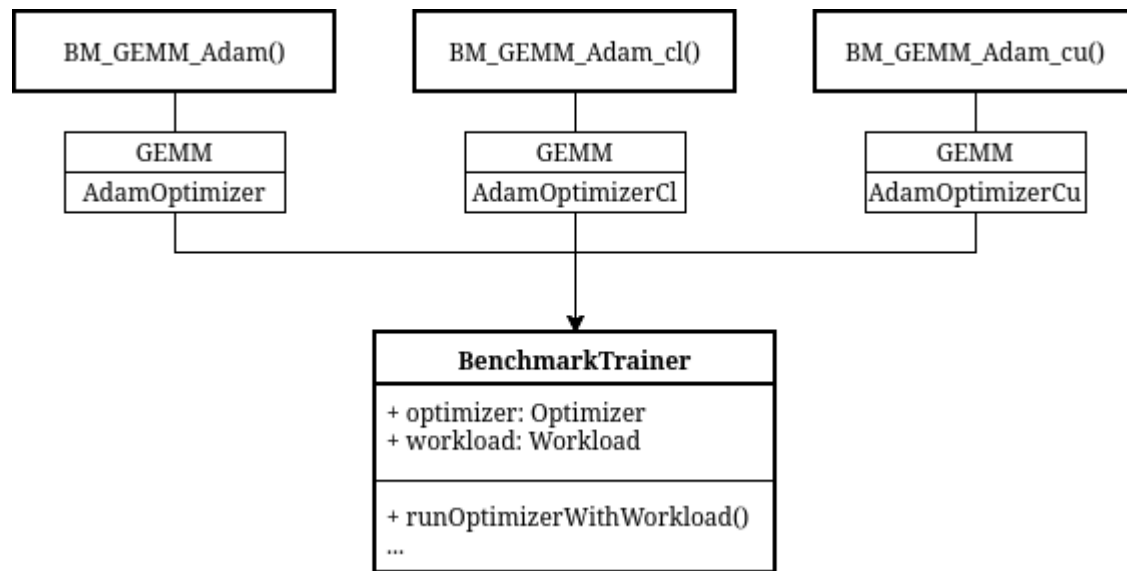
Benchmark Module

The entrypoint of the application

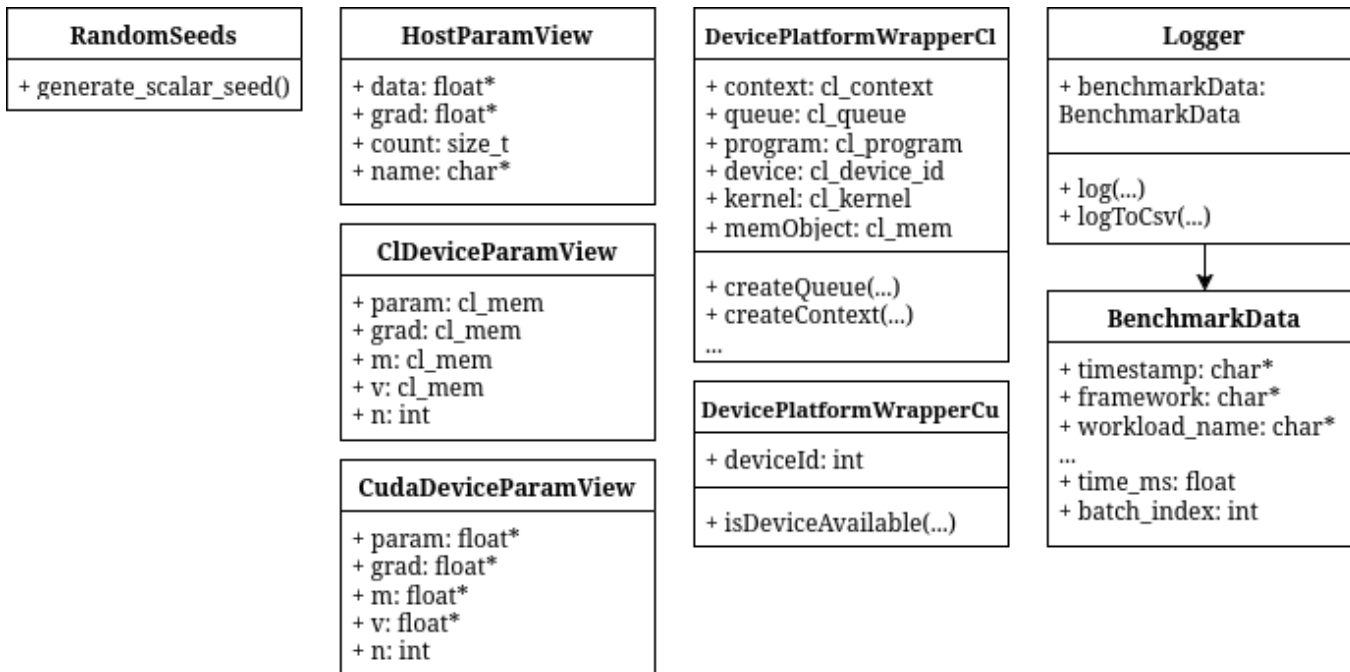


Benchmark Module

The entrypoint of the application



Util Module



A Review of measured Benchmarks

Measured aspects in consideration

Metric

Computation-Time

Data-Transfer-time

Quality-of-solution

Build-time

Measurement

Identify the execution of the kernel, and measure time difference.

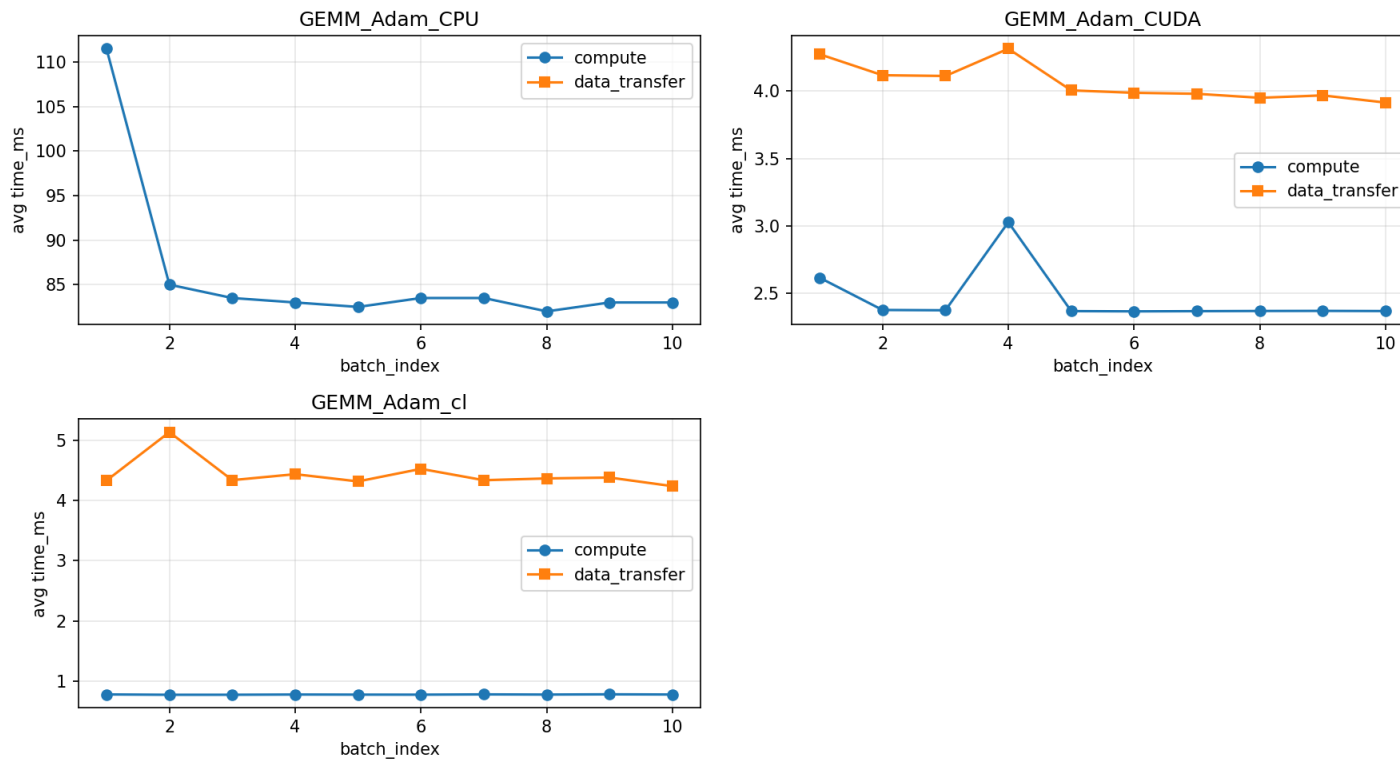
Identify the transfer from Host to Device and measure time difference.

Compare the 10th batch run loss with expectation and normalize

Measure time to build kernel

Visuals of currently obtained measurements

Average Time by Batch Index



Conclusion so far

OpenCL and CUDA perform on average pretty similar...

...but some Remarks:

- Currently only tested on GTX 970 → old platform
- The number of results are sparse
- Some metrics are missing, e.g. build-time

Outlook

Some TODOs until end of march...

- × Platform independence
- × AdamW
- × More measurements

Sources

- [1] Rishi, Rahul. (2011). Improving the Character Recognition Efficiency of Feed Forward BP Neural Network. International Journal of Computer Science & Information Technology. 3. 10.5121/ijcsit.2011.3107.
- [2] Chi, Yujie & Schubert, Keith & Badal, Andreu & Roncali, Emilie. (2025). Review of GPU-based Monte Carlo simulation platforms for transmission and emission tomography in medicine. Physics in Medicine & Biology. 70. 10.1088/1361-6560/adfda7.
- [3] „From CUDA to OpenCL: Towards a Performance-portable Solution for Multi-platform GPU Programming“ Peng D., Luszczek P., Tomov S., Peterson G., Dongarra J. Parallel Computing, Volume 38, Issue 8, Pages 391-407, 2012
- [4] J. Fang, A. L. Varbanescu and H. Sips, "A Comprehensive Performance Comparison of CUDA and OpenCL," 2011 International Conference on Parallel Processing, Taipei, Taiwan, 2012, pp. 216-225, doi: 10.1109/ICPP.2011.45.
- [5] Karimi, Kamran. "A Performance Comparison of CUDA and OpenCL." Arxiv Preprint ArXiv:1005.2581, arxiv.org, 2010.
- [6] Holm, Havard Heitlo & Brodtkorb, André & Sætra, Martin. (2020). Performance and Energy Efficiency of CUDA and OpenCL for GPU Computing Using Python. 10.3233/APC200089.
- [7] Puya Memarzia, Farshad Khunjush „An In-depth Study on the Performance Impact of CUDA, OpenCL, and PTX Code“ Journal of Information and Computing Science Vol. 10, No. 2, 2015, pp.124-136
- [8] Aftab M. et al. "OpenCL Programming Guide" 2012 Pearson Education, Inc. ISBN-13: 978-0-321-74964-2
- [9] Kirk D. et al. "Programming Massively Parallel Processors" 2010 David B. Kirk/NVIDIA Corporation and Wen-mei Hwu. Published by Elsevier Inc. ISBN: 978-0-12-381472-2